

Jupyter Notebook Execution Report

Name: Your Name

Date: May 21, 2026

Cell 1: ■ Markdown

Final Project - Impact of AI on Students

This is a submission to the final project to the course Supervised Machine Learning: Classification by IBM on Coursera.

Cell 2: ■ Code

```
#The following packages need to be installed:
!pip install kagglehub[pandas-datasets]
!pip install scikit-learn
!pip install pandas
!pip install numpy
!pip install matplotlib
!pip install seaborn
```

Error:

Traceback (most recent call last):

```
File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 401,
    exec('\n'.join(lines[:-1]), glb)
File "<string>", line 2
    !pip install kagglehub[pandas-datasets]
    ^
```

SyntaxError: invalid syntax

During handling of the above exception, another exception occurred:

Traceback (most recent call last):

```
File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 408,
    exec(source, glb)
File "<string>", line 2
    !pip install kagglehub[pandas-datasets]
    ^
```

SyntaxError: invalid syntax

Cell 3: ■ Code

```
#For data handling and processing
import kagglehub
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder, LabelEncoder, StandardScaler,
PolynomialFeatures
```

```
#For plotting
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
sns.set_style('dark')
sns.set_theme('notebook')
sns.set_context('notebook')
```

```
#model_selection
from sklearn.model_selection import StratifiedShuffleSplit, GridSearchCV
```

```
#models
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import VotingClassifier
```

```
#metrics
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
```

Error:

Traceback (most recent call last):

```
File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 401,
  exec('\n'.join(lines[:-1]), glb)
File "<string>", line 9
  %matplotlib inline
  ^
```

SyntaxError: invalid syntax

During handling of the above exception, another exception occurred:

Traceback (most recent call last):

```
File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 408,
  exec(source, glb)
File "<string>", line 9
```

```
%matplotlib inline
^
SyntaxError: invalid syntax
```

Cell 4: ■ Markdown

Dataset Selection and Description

The dataset used in this notebook is the dataset "Impact of AI on Students" on Kaggle. As someone who recently graduated from university, I'm curious to see how AI usage affects current students. In particular, I will be studying the impact of AI on the risk of burnout among students.

The goal of this notebook is to determine whether or not AI-usage has a significant impact on risk of burnout among students.

Cell 5: ■ Markdown

Source

This data is publicly available on Kaggle. It can be found on the Kaggle website [here](https://www.kaggle.com/datasets/laveshjadon/ai-impact-on-students/data). We load it into a pandas dataframe.

Cell 6: ■ Code

```
# Import the latest version of the data into a pandas DataFrame
kagglehub.dataset_download("laveshjadon/ai-impact-on-students")

df = kagglehub.dataset_load(
    kagglehub.KaggleDatasetAdapter.PANDAS,
    "laveshjadon/ai-impact-on-students",
    'ai_student_impact_dataset (1).csv'
)
df.head()
```

Error:

```
Traceback (most recent call last):
  File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 401,
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 2, in <module>
NameError: name 'kagglehub' is not defined
```

Cell 7: ■ Markdown

Key Features and the Target

We are interested in the features related to AI. Those are the following:

| Feature Name | Description |

| ----- | ----- |

| Weekly_GenAI_Hours | Average hours per week spent using Generative AI tools |

| Primary_Use_Case | Main purpose for which the student uses AI |

| Prompt_Engineering_Skill | Self-assessed proficiency in crafting effective prompts |

| Tool_Diversity | Number of distinct AI tools used |

| Perceived_AI_Dependency | Student's self-rated dependency on AI tools |

| Paid_Subscription | Whether the student has a paid AI subscription |

The target variable is 'Burnout_Risk_Level'. This is the assessed burnout risk.

We are not interested in any of the other features.

Cell 8: ■ Code

```
#Eliminate all other features
feature_names = [
    'Weekly_GenAI_Hours',
    'Primary_Use_Case',
    'Prompt_Engineering_Skill',
    'Tool_Diversity',
    'Paid_Subscription',
    'Perceived_AI_Dependency'
]

df = df.drop([
    column for column in df.columns
    if column not in feature_names
    and column != 'Burnout_Risk_Level'
],
axis= 1
)
```

Error:

Traceback (most recent call last):

```

File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 401,
    exec('\n'.join(lines[:-1]), glb)
File "<string>", line 11
    df = df.drop([
        ^
SyntaxError: '(' was never closed
During handling of the above exception, another exception occurred:
Traceback (most recent call last):
  File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 408,
    exec(source, glb)
  File "<string>", line 11, in <module>
NameError: name 'df' is not defined

```

Cell 9: ■ Markdown

Analytical Approach and Model Development

We analyze the dataset. Then we train several models on this dataset.

Cell 10: ■ Markdown

Structure

We observe 50k rows, none of which have any missing values. However, there are some string-types and a boolean that will need to be encoded.

Cell 11: ■ Code

```
df.info()
```

Error:

```

Traceback (most recent call last):
  File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 403,
    result = eval(lines[-1], glb)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "<string>", line 1, in <module>
NameError: name 'df' is not defined

```

Cell 12: ■ Markdown

The following manipulations are required:

- The target Burnout_Risk_Level needs to be label-encoded
- Primary_Use_Case needs to be one-hot encoded

- Prompt_Engineering_Skill needs to ordinal-encoded
- Paid_Subscription needs to be converted to an integer

Cell 13: ■ Code

```
## label-encode the target
burnout_encoder = LabelEncoder()
df['Burnout_Risk_Level'] = burnout_encoder.fit_transform(df['Burnout_Risk_Level'])
```

Error:

```
Traceback (most recent call last):
  File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 401,
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 2, in <module>
NameError: name 'LabelEncoder' is not defined
```

Cell 14: ■ Code

```
## Convert Paid_Subscription to int
df['Paid_Subscription'] = df['Paid_Subscription'].astype(int)
```

Error:

```
Traceback (most recent call last):
  File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 403,
    result = eval(lines[-1], glb)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "<string>", line 1
    df['Paid_Subscription'] = df['Paid_Subscription'].astype(int)
    ^
SyntaxError: invalid syntax
During handling of the above exception, another exception occurred:
Traceback (most recent call last):
  File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 408,
    exec(source, glb)
  File "<string>", line 2, in <module>
NameError: name 'df' is not defined
```

Cell 15: ■ Code

```
## Label encode prompt-engineering skills
df['Prompt_Engineering_Skill'].unique()
```

Error:

```
Traceback (most recent call last):
```

```
File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 403,
    result = eval(lines[-1], glb)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "<string>", line 1, in <module>
NameError: name 'df' is not defined
```

Cell 16: ■ Code

```
prompt_engineering_encoder = OrdinalEncoder()

df['Prompt_Engineering_Skill'] =
prompt_engineering_encoder.fit_transform(df[['Prompt_Engineering_Skill']])
```

Error:

```
Traceback (most recent call last):
  File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 401,
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 1, in <module>
NameError: name 'OrdinalEncoder' is not defined
```

Cell 17: ■ Code

```
## One-hot-encode Primary_Use_Case using pandas get_dummies

df = pd.get_dummies(df, drop_first=True, dtype= int)
```

Error:

```
Traceback (most recent call last):
  File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 403,
    result = eval(lines[-1], glb)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "<string>", line 1
    df = pd.get_dummies(df, drop_first=True, dtype= int)
        ^
SyntaxError: invalid syntax
During handling of the above exception, another exception occurred:
Traceback (most recent call last):
  File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 408,
    exec(source, glb)
  File "<string>", line 2, in <module>
NameError: name 'pd' is not defined
```

Cell 18: ■ Markdown

We observe that every column is now numerical

Cell 19: ■ Code

```
df.info()
```

Error:

Traceback (most recent call last):

File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 403,

result = eval(lines[-1], glb)

^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

File "<string>", line 1, in <module>

NameError: name 'df' is not defined

Cell 20: ■ Code

```
df.describe().T
```

Error:

Traceback (most recent call last):

File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 403,

result = eval(lines[-1], glb)

^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

File "<string>", line 1, in <module>

NameError: name 'df' is not defined

Cell 21: ■ Markdown

Data Analysis

We start by plotting the distribution of the target.

Observe that the dataset is slightly skewed, but it's not drastic enough to warrant extensive correction.

Cell 22: ■ Code

```
df['Burnout_Risk_Level'].value_counts().plot.bar(color= ['red', 'green', 'orange'])
plt.xlabel('Risk of Burnout')
plt.ylabel('Frequency')
plt.title('Distribution of the risk of burnout among students')
plt.xticks([0, 1, 2], burnout_encoder.inverse_transform([0, 1, 2]));
```

Error:

Traceback (most recent call last):

File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 401,

exec('\n'.join(lines[:-1]), glb)

File "<string>", line 1, in <module>

NameError: name 'df' is not defined

Cell 23: ■ Markdown

We proceed to study feature-correlation.

We observe no features are correlated so much that we want to omit them. Of note is that perceived AI dependency is highly related with AI usage.

Cell 24: ■ Code

```
plt.figure(figsize=(10, 8))
corr = df.drop('Burnout_Risk_Level', axis=1).corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Heatmap')
plt.show()
```

Error:

Traceback (most recent call last):

File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 401,
exec('\n'.join(lines[:-1]), glb)

File "<string>", line 1, in <module>

NameError: name 'plt' is not defined

Cell 25: ■ Markdown

Splitting the Data

We use StratifiedShuffleSplit to preserve the distribution presented above.

Cell 26: ■ Code

```
X = df.drop('Burnout_Risk_Level', axis=1)
y = df['Burnout_Risk_Level']

sss = StratifiedShuffleSplit(test_size= 0.2, random_state=0)
train_idx, test_idx = next(sss.split(X, y))

X_train = X.iloc[train_idx]
X_test = X.iloc[test_idx]
y_train = y.iloc[train_idx]
y_test = y.iloc[test_idx]
```

Error:

Traceback (most recent call last):

```
File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 401,
    exec('\n'.join(lines[:-1]), glb)
```

```
File "<string>", line 1, in <module>
```

NameError: name 'df' is not defined

Cell 27: ■ Markdown

Fitting models

We fit different types of models, and compare them afterwards.

Cell 28: ■ Markdown

Decision Tree Classifier

We first try to fit a decision tree classifier, to see if we are able to overfit the data. It's often easier to regularize an overfit model than to start with regularizing. Note that we will be using accuracy as our metric of choice because this is fine with the dataset being this balanced.

Cell 29: ■ Code

```
dtc = DecisionTreeClassifier()
```

```
dtc.fit(X_train, y_train)
```

```
y_pred = dtc.predict(X_test)
```

```
print('Training accuracy:', accuracy_score(y_train, dtc.predict(X_train)))
```

```
print('Testing accuracy:', accuracy_score(y_test, y_pred))
```

Error:

Traceback (most recent call last):

```
File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 401,
    exec('\n'.join(lines[:-1]), glb)
```

```
File "<string>", line 1, in <module>
```

NameError: name 'DecisionTreeClassifier' is not defined

Cell 30: ■ Markdown

Because the train accuracy is way higher than the test accuracy, this model is clearly overfit. Now let's regularize the decision tree classifier to obtain better test results.

Cell 31: ■ Code

```

dtc_param_grid = {
    "max_depth": range(1, 21),
    "max_features": range(1, 10)
}

dtc_search = GridSearchCV(
    estimator= dtc,
    param_grid = dtc_param_grid,
    n_jobs= -1
)

dtc_search.fit(X_train, y_train)
best_dtc = dtc_search.best_estimator_

y_pred = best_dtc.predict(X_test)

print('Training accuracy:', accuracy_score(y_train, best_dtc.predict(X_train)))
print('Testing accuracy:', accuracy_score(y_test, y_pred))

```

Error:

```

Traceback (most recent call last):
  File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 401,
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 6, in <module>
NameError: name 'GridSearchCV' is not defined

```

Cell 32: ■ Code

```

#We verify that we choose the right values for hyperparameter optimization
dtc_search.best_params_

```

Error:

```

Traceback (most recent call last):
  File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 403,
    result = eval(lines[-1], glb)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "<string>", line 1, in <module>
NameError: name 'dtc_search' is not defined

```

Cell 33: ■ Markdown

We find an accuracy of approximately 0.5. This is better than guessing, but not extremely strong.

Cell 34: ■ Markdown

Logistic Regression

Let's try logistic regression.

Cell 35: ■ Code

```
lr_steps = [  
    ('ss', StandardScaler()),  
    ('lr', LogisticRegression())  
]  
  
lr_pipe = Pipeline(lr_steps)  
lr_pipe.fit(X_train, y_train)  
  
y_pred = lr_pipe.predict(X_test)  
  
print('Training accuracy:', accuracy_score(y_train, lr_pipe.predict(X_train)))  
print('Testing accuracy:', accuracy_score(y_test, y_pred))
```

Error:

```
Traceback (most recent call last):  
  File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 401,  
    exec('\n'.join(lines[:-1]), glb)  
  File "<string>", line 2, in <module>  
NameError: name 'StandardScaler' is not defined
```

Cell 36: ■ Markdown

This model isn't powerful enough in its standard form. Let's try adding polynomial features and changing the amount of regularization.

Cell 37: ■ Code

```
lr_steps = [  
    ('pf', PolynomialFeatures(include_bias= False)),  
    ('ss', StandardScaler()),  
    ('lr', LogisticRegression(max_iter= 1000))  
]  
  
lr_pipe = Pipeline(lr_steps)
```

```

lr_param_grid = {
    'pf__degree': range(1, 4),
    'lr__C' : [0.01, 0.1, 1.0, 10.0, 100.0]
}

lr_search = GridSearchCV(
    estimator= lr_pipe,
    param_grid= lr_param_grid,
    n_jobs= -1
)

lr_search.fit(X_train, y_train)
best_lr = lr_search.best_estimator_

y_pred = best_lr.predict(X_test)

print('Training accuracy:', accuracy_score(y_train, best_lr.predict(X_train)))
print('Testing accuracy:', accuracy_score(y_test, y_pred))

```

Error:

```

Traceback (most recent call last):
  File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 401,
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 2, in <module>
NameError: name 'PolynomialFeatures' is not defined

```

Cell 38: ■ Code

```

#We verify that we choose the right values for hyperparameter optimization

lr_search.best_params_

```

Error:

```

Traceback (most recent call last):
  File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 403,
    result = eval(lines[-1], glb)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "<string>", line 1, in <module>
NameError: name 'lr_search' is not defined

```

Cell 39: ■ Markdown

Adding polynomial features did pretty much nothing. The results after adding polynomial features and tuning the regularization are pretty much identical.

We find an accuracy of approximately 0.5. This is better than guessing, but not extremely strong.

Cell 40: ■ Markdown

K nearest Neighbours

We don't have a lot of features, so K nearest is a good model to try as we are not suffering from the curse of dimensionality.

Cell 41: ■ Code

```
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)

print('Training accuracy:', accuracy_score(y_train, knn.predict(X_train)))
print('Testing accuracy:', accuracy_score(y_test, y_pred))
```

Error:

```
Traceback (most recent call last):
  File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 401,
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 1, in <module>
NameError: name 'KNeighborsClassifier' is not defined
```

Cell 42: ■ Markdown

This is better than the logistic regression, so maybe we can find a better model here.

Cell 43: ■ Code

```
knn_param_grid = {
    'n_neighbors' : range(1, 300, 4)
}

knn_search = GridSearchCV(
    estimator= knn,
    param_grid = knn_param_grid,
    n_jobs= -1
)
```

```
knn_search.fit(X_train, y_train)
best_knn = knn_search.best_estimator_

y_pred = best_knn.predict(X_test)

print('Training accuracy:', accuracy_score(y_train, best_knn.predict(X_train)))
print('Testing accuracy:', accuracy_score(y_test, y_pred))
```

Error:

```
Traceback (most recent call last):
  File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 401,
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 5, in <module>
NameError: name 'GridSearchCV' is not defined
```

Cell 44: ■ Code

```
#We verify that we choose the right values for hyperparameter optimization
knn_search.best_params_
```

Error:

```
Traceback (most recent call last):
  File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 403,
    result = eval(lines[-1], glb)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "<string>", line 1, in <module>
NameError: name 'knn_search' is not defined
```

Cell 45: ■ Markdown

We seem to get an accuracy of 0.5 again. Still better than guessing.

Cell 46: ■ Markdown

Support Vector Machine

The last model covered in this course is SVM. We try that here.

Cell 47: ■ Code

```
svc = SVC()
svc.fit(X_train, y_train)

y_pred = svc.predict(X_test)
```

```
print('Training accuracy:', accuracy_score(y_train, svc.predict(X_train)))
print('Testing accuracy:', accuracy_score(y_test, y_pred))
```

Error:

```
Traceback (most recent call last):
  File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 401,
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 1, in <module>
NameError: name 'SVC' is not defined
```

Cell 48: ■ Markdown

We immediately obtain equal accuracy without any hyperparameter tuning. We still try our best to improve, but it's unlikely it will work.

Cell 49: ■ Code

```
svc_param_grid = {
    "C" : [0.1, 1, 10]
}

svc_search = GridSearchCV(
    estimator= svc,
    param_grid= svc_param_grid,
    n_jobs= -1
)

svc_search.fit(X_train, y_train)
best_svc = svc_search.best_estimator_

y_pred = best_svc.predict(X_test)

print('Training accuracy:', accuracy_score(y_train, best_svc.predict(X_train)))
print('Testing accuracy:', accuracy_score(y_test, y_pred))
```

Error:

```
Traceback (most recent call last):
  File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 401,
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 5, in <module>
NameError: name 'GridSearchCV' is not defined
```


Cell 50: ■ Markdown

We obtain an accuracy of 0.5, which is identical to that of our models.

Cell 51: ■ Markdown

Stacking all the models together

Let's see what happens when we stack all the models together.

Cell 52: ■ Code

```
estimators = [  
    ('dtc', best_dtc),  
    ('lr', best_lr),  
    ('knn', best_knn),  
    ('svc', best_svc)  
]  
  
stack = VotingClassifier(  
    estimators= estimators  
)  
stack.fit(X_train, y_train)  
  
y_pred = stack.predict(X_test)  
  
print('Training accuracy:', accuracy_score(y_train, stack.predict(X_train)))  
print('Testing accuracy:', accuracy_score(y_test, y_pred))
```

Error:

```
Traceback (most recent call last):  
  File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 401,  
    exec('\n'.join(lines[:-1]), glb)  
  File "<string>", line 2, in <module>  
NameError: name 'best_dtc' is not defined
```

Cell 53: ■ Markdown

It seems that stacking all models still yields an accuracy of 0.5.

Cell 54: ■ Markdown

Comparing different models

We give different error metrics of all models, to see if a certain model might be better to predict who has a high risk of burn-out than others.

Cell 55: ■ Code

```
performance_overview = pd.DataFrame()

models = [
    ('dtc', best_dtc),
    ('lr', best_lr),
    ('knn', best_knn),
    ('svc', best_svc),
    ('stack', stack)
]

for model in models:
    y_pred = model[1].predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    precision, recall, f1, _ = precision_recall_fscore_support(y_test, y_pred)

    score_overview = {
        'accuracy': accuracy,
        'precision_high_risk': precision[0],
        'recall_high_risk': recall[0],
        'f1_high_risk': f1[0]
    }

    performance_overview[model[0]] = score_overview

performance_overview
```

Error:

```
Traceback (most recent call last):
  File "/home/reinder/.vscode/extensions/ganeshkumbhar.nb2pdf-1.1.9/scripts/nb2pdf.py", line 401,
    exec('\n'.join(lines[:-1]), glb)
  File "<string>", line 1, in <module>
NameError: name 'pd' is not defined
```

Cell 56: ■ Markdown

The models perform very similar, but some useful findings:

- logistic regression and support vector machine have the highest precision.
- knn and dtc have the highest recall.
- the stack is somewhere in between.

Cell 57: ■ Markdown

Conclusions

All models have an accuracy of around 0.5. Given that there are 3 classes, this is significantly better than guessing!

Hence, we conclude that there is a significant correlation between AI usage among students and risk of burnouts.

Cell 58: ■ Markdown

Use of these models

If your goal is to identify as many students as possible with a high risk of burnout, I'd recommend the models Decision Tree Classifier or Support Vector Machine. These models find 45% of all students with a high risk of burnout.

If your goal is to have measures taken to help people with a high risk of burnout be as likely as possible to help someone who actually has a high risk, I'd recommend the models Logistic Regression or K Nearest Neighbors. These models are right 66% percent of the time when identifying a student as 'high risk of burnout'.

Cell 59: ■ Markdown

Limitations and Flaws

The models in this notebook are trained only on data of AI-usage of students. This was done to study if a relation between AI-usage and burnout can be found. It is evident that such a relationship exists.

However, these models are not great for pinpointing students with a high risk of burnout. Obviously, there are more factors to burnout than AI-usage.

Cell 60: ■ Markdown

Improvements and Next steps

One way to find a better way of pinpointing these students is train the models on the entire Kaggle dataset. This would yield stronger models, that are more useful in the real world. To achieve this, one would retrain all the models mentioned above on the entire dataset, and find which model performs best.

Another improvement that could be is to properly interpret these models. By means of our error-measuring before, we have done some interpretation already. However, more could be done. One possible way to interpret these models is to use LIME. This would be my starting point.